Sistemas microprogramables

Lógicas cableada y programada

En electrónica digital se pueden diferenciar dos lógicas:

- Lógica cableada.
- Lógica programada.

- **Lógica cableada:** En este tipo de lógica, para cada aplicación ha de diseñarse un circuito específico, que luego es muy difícil (o imposible) de modificar, se basa en compuertas lógicas (hardware).
- **Lógica programada:** En este tipo de lógica un mismo dispositivo o conjunto de dispositivos pueden resolver infinidad de problemas gracias a que pueden configurarse de distintas maneras programando su funcionamiento, se basa en dispositivos programables (software).

Definición de un sistema microprogramable

En términos generales un sistema microprogramable es un dispositivo o conjunto de dispositivos de propósito general, que según sea necesario se programan para resolver distintos problemas.

Tienen una gran variedad de aplicaciones, ya que simplemente variando la programación, se les puede indicar que realicen una función u otra, siendo las más importantes:

- **Aplicaciones informáticas:** Los ordenadores personales son sistemas microprogramables en los que el usuario introduce el programa de aplicación que desea utilizar en cada momento. También se encuentran en casi todos los periféricos; ratones, teclados, impresoras, escáner, etc.
- **Cálculo matemático:** Las modernas calculadoras y los grandes ordenadores de cálculo.
- **Procesos industriales:** Los sistemas microprogramables, en numerosas ocasiones, se utilizan para controlar procesos industriales, como es el caso de los autómatas programables (PLC) utilizados para controlar robots, cadenas de montaje, etc.
- **Electrodomésticos:** lavadoras, hornos, frigoríficos, lavavajillas, batidoras, televisores, vídeos, reproductores DVD, equipos de música, mandos a distancia, consolas, etc.
- **Sistemas de comunicación:** en grandes automatismos como centrales y en teléfonos fijos, móviles, fax, etc.
- **Automoción:** climatización, seguridad, ABS, etc.
- **Sistemas de supervisión, vigilancia y alarma:** ascensores, calefacción, aire acondicionado, alarmas de incendio, robo, etc.
- **Otros:** Instrumentación, electromedicina, tarjetas de identificación y control de acceso (smartcard, SUBE, Monedero, Etc.), sistemas de navegación, etc.

Clasificación de los sistemas microprogramables

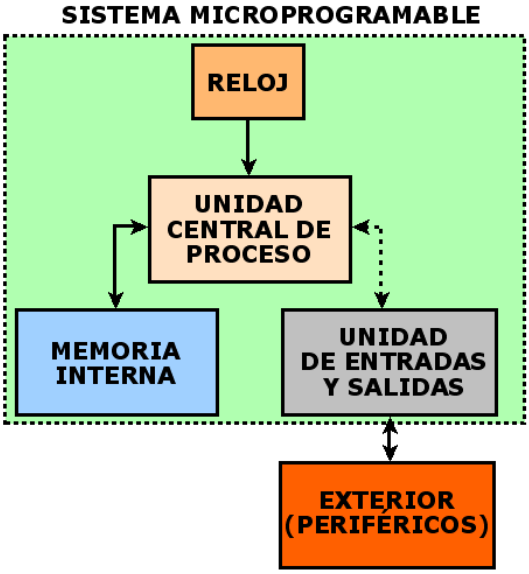
Podemos clasificar los sistemas microprogramables según el número de distintos circuitos integrados que lo forman, la capacidad de trabajo y el tratamiento de datos en:

- Microprocesadores
- Microcontroladores
- Dispositivos PLDs

Microprocesadores y microcontroladores

Estos sistemas son capaces de leer, interpretar y luego ejecutar de forma secuencial las instrucciones contenidas en un programa.

Todos los sistemas microprogramables por complejos que sean (salvo los PLDs) poseen la estructura que se muestra en la siguiente figura.



En dicha estructura se distinguen los siguientes elementos:

Reloj: Es un generador de ondas cuadradas periódicas, utilizado para marcar el ritmo de trabajo y que todo el sistema esté sincronizado.

Unidad Central de Proceso o CPU (Central Process Unit): Es la parte más importante del sistema microprogramable. Es donde se realiza la interpretación y ejecución de las instrucciones, se generan todas las órdenes de control para gobernar todo el sistema y se realizan las operaciones aritméticas y lógicas. Todo ello se realiza con los datos procedentes de la Memoria Central o los contenidos en registros ordenadas desde la Unidad de Control. También, es la encargada de realizar todas las transferencias de datos hacia la memoria o desde esta.

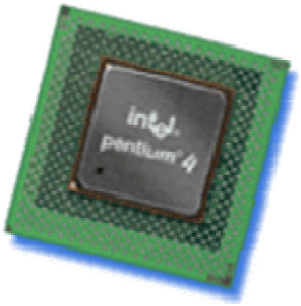
Todas sus funciones se realizan en sincronía con la señal del reloj, por ello, la frecuencia del reloj define la velocidad del sistema. Su estructura interna es compleja y si se presenta todo integrado en un único circuito integrado se denominada microprocesador. El uso de dichos microprocesadores está muy extendido y son ampliamente utilizados en sistemas tales como ordenadores personales, controles industriales, grandes ordenadores o mainframes, etc.

Memoria Central o Interna: En este tipo de dispositivos se encuentran los datos y programas que debe utilizar el sistema microprogramable (BIOS en el caso de una PC). Existe otro tipo de memorias con las cuales no se deben confundir, denominadas memorias de masa, que forman parte de los periféricos y se encuentran fuera del sistema, como por ejemplo los discos duros en las PC.

Unidad de entrada/salida (interface): Este circuito permite la comunicación del sistema microprogramable con el exterior. Su función fundamental es la de adaptar las diferentes velocidades y códigos utilizados por los elementos externos del sistema y el interior.

Periféricos: Estrictamente hablando podríamos decir que no forman parte del sistema microprogramable. Es un conjunto de dispositivos que realizan un trabajo en el exterior del sistema. Estos periféricos pueden ser de entrada o de salida, aunque existen algunos que realizan ambas. Por ejemplo, en un PC los periféricos de entrada más importantes son el teclado y el ratón y los de salida el monitor y la impresora.

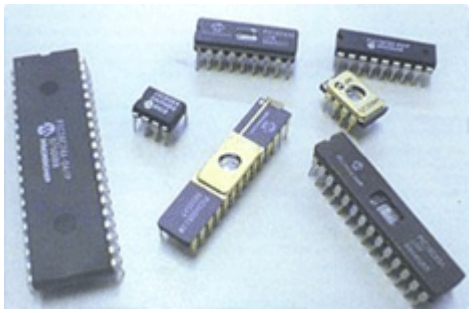
Microprocesadores



El sistema microprogramable está dividido en diversos circuitos integrados, denominándose al más importante microprocesador. El microprocesador incluye únicamente en su interior la Unidad Central de Proceso, de ahí, que se le conozca también como **CPU**.

Los microprocesadores se utilizan en aquellas aplicaciones en las que se requieren grandes cantidades de memoria, ejecutar una gran cantidad de programas o se prevea hacer futuras ampliaciones realizando las tareas a una elevada velocidad. Un ejemplo de aplicación son los ordenadores personales.

Microcontroladores



Todo el sistema microprogramable está integrado en un único circuito integrado, de manera que un microcontrolador es un circuito integrado que integra en la misma pastilla la CPU, la Memoria Central, los puertos de E/S digitales, algunos periféricos y otros elementos. Como desventaja su capacidad de trabajo es menor que la de los microprocesadores. Se utiliza en aquellas aplicaciones en las que necesitamos el uso de un pequeño procesador. Se encuentra por todas partes, por ejemplo, los autómatas programables industriales poseen este tipo de circuitos integrados, al que se le han añadido sistemas de control de potencia tales como relés para poder controlar maquinaria industrial.

Dispositivos PLDs

Los PLDs (Programmable Logic Device) son circuitos integrados que en su interior integran una matriz de puertas lógicas cuya funcionalidad puede ser programada por el usuario. Internamente no tienen la estructura de microprocesadores y microcontroladores pues no están basados en una CPU que lea un programa de una memoria. Esto significa que no pueden sustituir a los sistemas basados en CPU en muchas aplicaciones y además no son tan versátiles. No obstante existen aplicaciones donde son más adecuados que estos, como por ejemplo para resolver problemas simples puramente combinacionales.

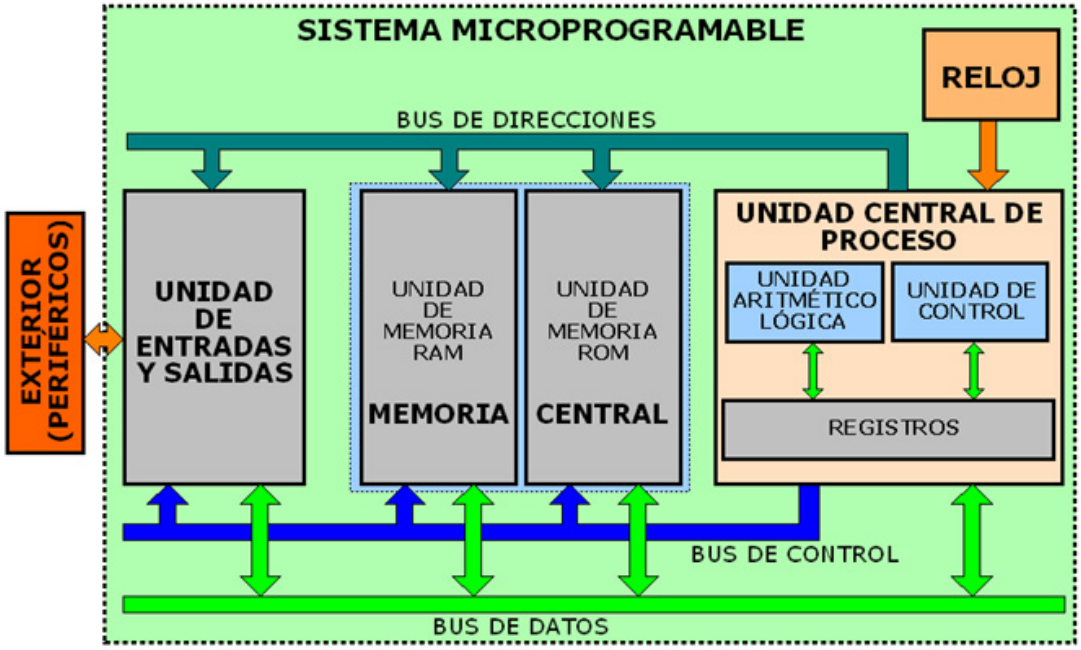
Hardware y software

Un sistema microprogramable está formado por los siguientes componentes:

- **Hardware:** Es el conjunto de circuitos electrónicos que forman el sistema microprogramable o, dicho de otra forma, es la parte física del sistema.
- **Software:** Es el conjunto de programas y aplicaciones formado por instrucciones y rutinas que se utilizan para programar y coordinar al sistema microprogramable. También se denomina software al conjunto de lenguajes empleados para elaborar dichos programas.
- **Firmware:** Es un "software" grabado en la estructura electrónica del sistema microprogramable y que el usuario en principio no puede alterar. En los ordenadores personales, la BIOS (Basic Input/Output System) utiliza firmware y contiene un grupo de programas que sirven de intermediario entre el software y hardware.

Arquitectura de un sistema basado en CPU

En este punto veremos con mayor profundidad cuales son los bloques que componen un sistema microprogramable basado en el uso de una CPU. Esto incluye tanto a los sistemas basados en microprocesador como los que utilizan microcontroladores.



Elementos esenciales que constituyen un sistema basado en CPU:

- **Reloj**
- **CPU**
  - **Unidad Aritmética-Lógica (ALU).**
  - **Acumuladores y Registros.**
  - **Unidad de Control (UC).**
- **Memoria**
- **Unidad de entrada/salida**
- **Buses del sistema**

Reloj

Para que el microprocesador genere todas las señales necesarias para controlar los restantes bloques del sistema y para que todo el sistema esté sincronizado (que cada elemento funcione cuando le corresponde), se parte de ondas cuadradas de frecuencia constante generadas normalmente por un cristal de cuarzo, aunque también se utilizan osciladores RC.

A partir de esta señal de reloj, se genera el denominado ciclo máquina, que está formado por dos fases:

- **Fase de búsqueda (Fetch cycle):** En esta fase se realiza la búsqueda de una instrucción en memoria y se guarda en el registro correspondiente.
- **Fase de ejecución (Execute cycle):** En esta fase se ejecuta o realiza la transferencia de datos ordenada.

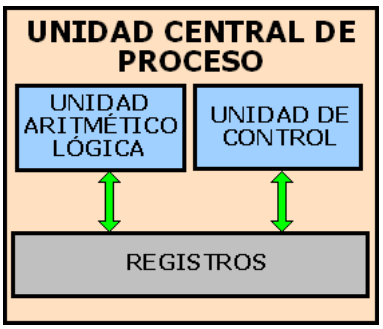
Este ciclo máquina tarda en realizarse más o menos tiempo dependiendo del fabricante, En el caso de un PIC, cada ciclo de maquina demanda 4 ciclos de reloj, es decir que la frecuencia de trabajo del sistema es: Frec. clock / 4

A veces para completar la ejecución de una instrucción se requiere más de un ciclo máquina.

CPU

La CPU está formada por:

- Unidad Aritmética-Lógica
- Acumuladores y Registros
- Unidad de Control

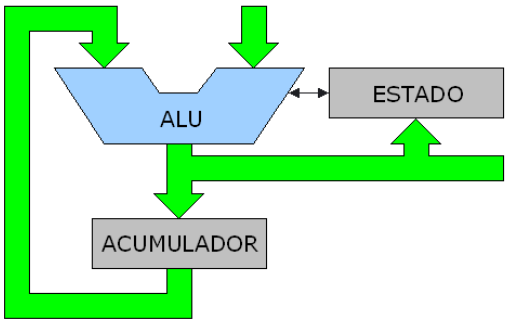


Unidad Aritmética Lógica ALU (Aritmetic Logic Unit)

Está compuesta por un circuito combinacional complejo que se encarga de realizar las operaciones aritméticas (suma, multiplicación...), lógicas (AND, OR...), desplazamientos, etc.

Estas operaciones son ordenadas por las instrucciones que se están ejecutando, con las informaciones presentes en los registros de entrada y/o en posiciones determinadas de la Memoria Central y devuelve los resultados a los registros o a la Memoria Central.

Como se utiliza: Normalmente uno de los datos de entrada proviene del registro acumulador (ACUM) y el otro de un registro interno, una posición de memoria o como literal de la propia instrucción. El resultado se devuelve en el acumulador machacando el valor que tenía antes. Hay también otros tipos de funcionamiento.



Acumuladores y registros

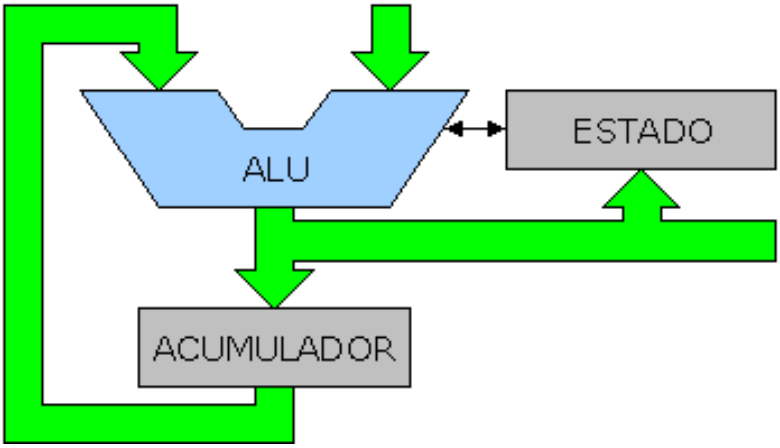
Un registro es una pequeña memoria interna, donde se almacenan temporalmente los resultados intermedios de las operaciones. Los registros están formados por biestables colocados de tal forma que pueden almacenar la información en un momento determinado. La longitud de estos registros viene determinada por el número de biestables que los forman.

Existen varios registros diferentes, entre los que destacamos:

- Registro acumulador.
- Registro de estado.
- Registros auxiliares.
- Registro SP (Stack Pointer).
  - La pila.
- Registro CP (Contador de Programa)

Registro acumulador

Como hemos mencionado, en la ALU se realizan las operaciones aritméticas y lógicas con los datos.



La CPU contiene al menos un acumulador o registro en el que se almacenan datos procedentes de la memoria o de otros registros que posea el sistema. Las operaciones con dos comandos se realizan a través de la Unidad Aritmética-Lógica (ALU) y siempre entre:

- acumuladores (si existe más de uno)
- entre el acumulador y el contenido de una posición de memoria
- entre un acumulador y otro registro
- entre el acumulador y un término constante implícito en la propia instrucción

Es decir, siempre las operaciones se realizan a través del acumulador, apareciendo también el resultado en el acumulador.

Registro de estado (status)

En una CPU, existen unos biestables denominados banderas o **"flags"** que cambian de estado como consecuencia del resultado de la última operación efectuada sobre el registro acumulador. La estructura del registro de estado varía mucho de unos sistemas a otros, tanto en la posición que cada uno de los flag ocupa dentro del registro, como en los tipos de informaciones que almacenan. A continuación se muestran algunas informaciones que puede proporcionar el registro de estado en una CPU:

- **OF (*Overflow Flag o* Bandera de Desbordamiento):** Se pone a uno si se produce un desbordamiento. Es decir, el tamaño del resultado de una operación es mayor que el lugar donde debe guardarse. Su contenido puede ser modificado por software.
- **SF (*Sign Flag o* Bandera de Signo):** Nos indica el signo del resultado de una operación. Si el resultado es negativo se pone a uno, y a cero si es positivo.
- **ZF (*Zero Flag o* Bandera de Cero):** Se activa si el resultado de una operación es 0. Su estado no es modificable por *software.*
- **PF (*Parity Flag o* Bandera de Paridad):** Se activa (pone a uno), si el resultado de una operación es un número par.
- **CF (*Carry Flag o* Bandera de Acarreo):** Se pone a 1, si existe acarreo final en el *bit 7* en operaciones de 8 *bits (byte) y* en el *bit* 15 en operaciones de 16 *bits* (palabra). También es utilizado en los desplazamientos.
- **IF (*Interrupt Flag o* Bandera de Interrupciones):** Las interrupciones son peticiones de parada a la *CPU,* que llegan desde el exterior a través de ciertas líneas del bus de control. Cuando se encuentra a 1, permite las interrupciones externas *(hardware)* que llegan al microprocesador.
- **AF (*Auxiliary Flag* o Acarreo Auxiliar):** Se pone a uno, si hay acarreo parcial del *bit 3* al 4 en operaciones con 8 *bits (byte,* aritmética BCD) o si hay acarreo parcial del *bit 7* al 8 en operaciones con 16 *bits* (palabra).
- **DF (*Decimal Flag o* Bandera decimal):** Cuando este *flag* está a uno, la *ALU* realiza las operaciones aritméticas en BCD y, si está a 0, opera en binario natural.

En la mayoría de las CPU existen bits en este registro que no contiene información alguna y, en general, están a 1.

Registro de estado en el PIC16F88

A modo de ejemplo se muestra el registro de Estado (STATUS) del microcontrolador PIC16F88, que contiene varios bits de estado de la unidad central, el estado aritmético de la ALU, el estado del reset y un par de bits de selección de página denominados RP1 y RP0. Ocupa la posición 03h del banco 0, la 83h, 103h, 183h, de los bancos 1, 2 y 3 respectivamente.

R/W-0	R/W-0	R/W-0	R-1	R-1	R/W-x	R/W-x	R/W-x
IRP	RP1	RP0	TO	PD	Z	DC	C
bit 7							bit 0

- **IRP**: Selección del banco en direccionamiento indirecto. Este bit junto con el de más peso del registro FSR sirven para determinar el banco de la memoria de datos seleccionado.
- **RP0** y **RP1**: Register Bank Select. Selección de página o banco de la memoria con direccionamiento directo. Cada página contiene un máximo de 128 bytes. El valor binario representado por estos dos bits indica el banco que se está direccionando.
- **TO**: Time Out
  - 1. Se pone a 1 tras conectar la alimentación o al ejecutar la instrucción CLRWDT o SLEEP.
  - 0. Se pone a 0 por desbordamiento del Watch Dog Timer.
- **PD**: Power Down.
  - 1. Se pone automáticamente a 1 tras conectar la alimentación o al ejecutar la instrucción CLRWDT.
  - 0. Se pone a 0 al ejecutar la instrucción SLEEP.
- **Z**: Cero
  - 1 = El resultado de la última operación aritmética o lógica fue 0.
  - 0 = El resultado fue distinto de 0.
- **DC** (Digit Carry). Acarreo en el 4º bit de menos peso. Funciona igual que el bit de Carry descrito a continuación. De interés en operaciones en BCD (4 bits)
- **C** (Carry). Acarreo en el 8º bit o bit de más peso. Es el bit de "acarreo" en operaciones de suma ADDWF y ADDLW así como también el bit de "llevada" en las instrucciones de sustracción SUBWF y SUBLW. También lo emplean las instrucciones RLF y RRF de rotación de bits.
  - Suma
    - 1. Se pone a 1 cuando se ha producido acarreo en la suma en el bit de mayor peso con las operaciones ADDWF y ADDLW.
    - 0. Se pone a 0 si en la suma no se ha producido acarreo.
  - Resta
    - 1. Se pone a 1 si en la resta no se ha producido desborde (resultado positivo).
    - 0. Se pone a 0 cuando se ha producido desborde en la resta con las operaciones SUBWF y SUBLW (resultado negativo). Esto es así debido a que la resta se ejecuta sumando el complemento A2 del número.

Registros auxiliares

Estos registros auxiliares o de uso general los puede utilizar el programador para almacenar datos o resultados intermedios. El número de estos registros depende del tipo de microprocesador.

Registro Stack Pointer

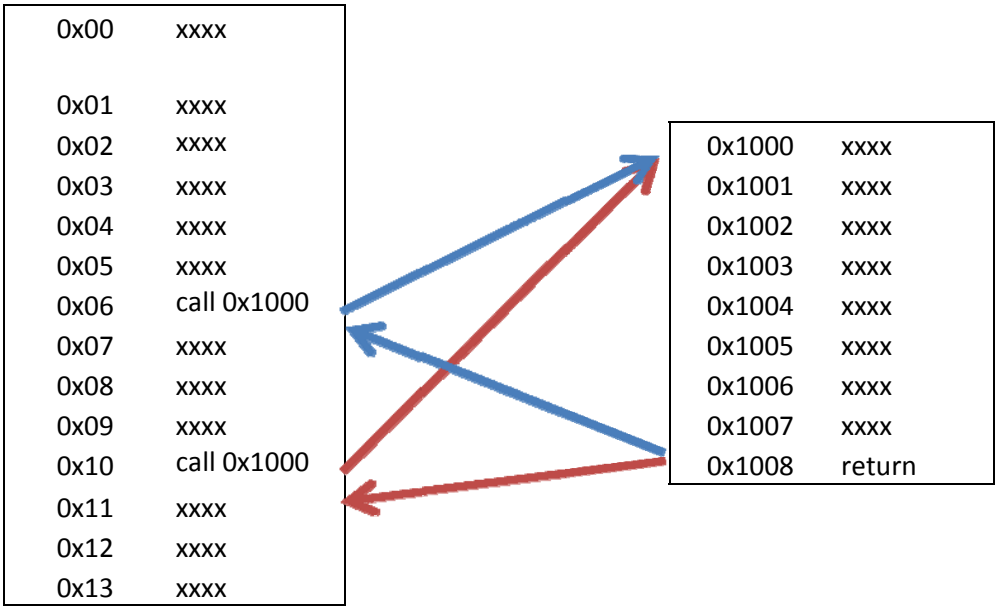
En los microcontroladores PIC este registro **NO** es accesible ni para lectura ni para escritura, y apunta a una zona de memoria denominada pila o Stack. De ahí, su nombre de puntero de pila o Stack Pointer, se utiliza para realizar saltos de programa, llamar subrutinas, etc.

La pila o Stack

La pila en los ordenadores personales es una zona de memoria, que es una parte de la memoria central (RAM), de manera que se reserva un pequeño espacio de esta para que funcione como una pila, denominándolo segmento de pila. Sin embargo, por ejemplo en los microcontroladores PIC, se encuentra separada tanto de la memoria de programa como de la de datos y está formada por registros. En todo caso, para moverse por la pila, almacenar y recuperar los datos se utiliza el registro puntero de pila o Stack pointer.

La pila tiene una estructura denominada LIFO (Last In First Out), por lo que el último valor que se guarda es el primero que sale. Este tipo de estructura gestiona la entrada y salida de información del mismo modo que si actuara manipulando una pila de papeles, donde el último papel en apilarse es el que luego primero se recoge.

La pila se utiliza para almacenar el valor del contador de programa con el fin de realizar un salto y poder continuar luego ejecutando el programa desde donde se realizo el salto.



Normalmente el sistema dispone de una bandera (flag o identificador) que indica un desbordamiento de la pila, si bien hay casos, como en la mayoría de los microcontroladores PIC16 donde no se dispone de esta bandera y hay que tenerlo en cuenta en el programa que se diseñe.

Registro Contador de Programa

Es un registro que se encuentra en la Unidad de Control y que indica en qué posición de la memoria se encuentra la próxima instrucción a ejecutar.

Este registro, se incrementa automáticamente para obtener las instrucciones y los datos de la memoria de modo secuencial.

Unidad de Control (Control Unit, CU)

Es una parte fundamental del sistema puesto que se encarga de gobernar el funcionamiento global del mismo. Recibe la información, la transforma e interpreta, enviando las órdenes precisas a los elementos que las requieren para un procesamiento correcto de los datos.

En la Unidad de Control se encuentra el **contador de programa (CP)** que indica en qué posición de la memoria se encuentra la próxima instrucción a ejecutar.

En esta unidad se encuentra también el **decodificador de instrucciones (DI)**, dispositivo que "traduce" las instrucciones del programa contenidas en la memoria a micro-ordenes grabadas internamente, para saber los pasos que debe realizar para ejecutar la instrucción.

Una vez decodificada la instrucción, la Unidad de Control generará todas las señales de control necesarias para hacer funcionar al resto de elementos del sistema y que todo funcione en orden.

Memoria (memoria central o principal)

Es la encargada del almacenamiento de los programas y la información necesaria para el funcionamiento del sistema. Se compone de celdas o palabras de memoria.

Su función en el sistema es la de almacenar los programas a ejecutar, los datos y los resultados intermedios del proceso.

No debemos confundirla con las memorias de almacenamiento masivo que se encuentran en los periféricos de entrada/salida, como los discos duros o las memorias extraíbles de un PC.

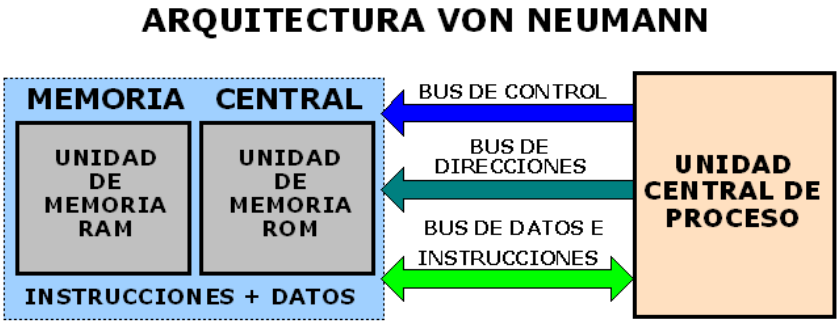
La Memoria suele dividirse en:

- **Memoria de programa:** Es la zona de memoria donde se almacenan los programas a ejecutar. No se modifica durante la ejecución de un programa, ya que en ella se guardan las instrucciones que lo forman.
- **Memoria de datos o de trabajo:** En esta zona se almacenan los datos del programa a ejecutar. Siempre es una RAM puesto que se modifica durante la ejecución de un programa, ya que en ella se almacenan y se leen los resultados de las operaciones realizadas, así como los datos de las operaciones intermedias requeridas.

Arquitectura Von Neumann y arquitectura Harvard

Hay dos arquitecturas distintas relacionadas con el uso y distribución de la memoria:

Arquitectura Von Neumann: Tradicionalmente los sistemas con microprocesadores se basan en esta arquitectura, en la cual la unidad central de proceso (CPU), está conectada a una memoria principal única (casi siempre sólo RAM) donde se guardan las instrucciones del programa y los datos. A dicha memoria se accede a través de un sistema de buses único (control, direcciones y datos):



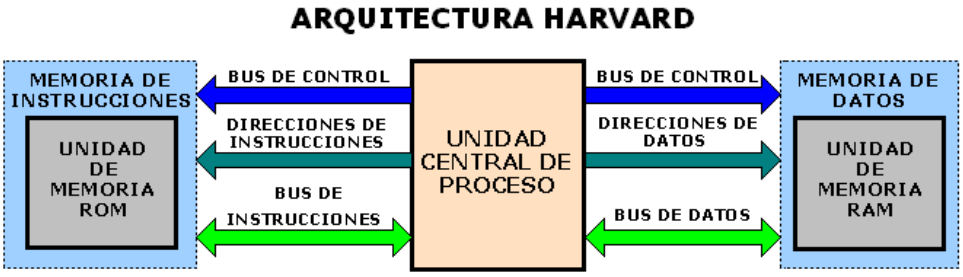
En un sistema con arquitectura Von Neumann el tamaño de la unidad de datos o instrucciones está fijado por el ancho del bus que comunica la memoria con la CPU. Así un microprocesador de 8 bits con un bus de 8 bits, tendrá que manejar datos e instrucciones de una o más unidades de 8 bits (bytes) de longitud. Si tiene que acceder a una instrucción o dato de más de un byte de longitud, tendrá que realizar más de un acceso a la memoria.

El tener un único bus hace que el microprocesador sea más lento en su respuesta, ya que no puede buscar en memoria una nueva instrucción mientras no finalicen las transferencias de datos de la instrucción anterior.

Las principales limitaciones que nos encontramos con la arquitectura Von Neumann son:

- La limitación de la longitud de las instrucciones por el bus de datos, que hace que el microprocesador tenga que realizar varios accesos a memoria para buscar instrucciones complejas.
- La limitación de la velocidad de operación a causa del bus único para datos e instrucciones que no deja acceder simultáneamente a unos y otras, lo cual impide superponer ambos tiempos de acceso.

Arquitectura Harvard: Este modelo, que utilizan los microcontroladores PIC, tiene la unidad central de proceso (CPU) conectada a dos memorias (una con las instrucciones y otra con los datos) por medio de dos buses diferentes.



Una de las memorias contiene solamente las instrucciones del programa (Memoria de Programa), y la otra sólo almacena datos (Memoria de Datos).

Ambos buses son totalmente independientes lo que permite que la CPU pueda acceder de forma independiente y simultánea a la memoria de datos y a la de instrucciones. Como los buses son independientes éstos pueden tener distintos contenidos en la misma dirección y también distinta capacidad. También la longitud de los datos y las instrucciones puede ser distinta, lo que optimiza el uso de la memoria en general.

Para un procesador de Set de Instrucciones Reducido, o RISC (Reduced Instrucción Set Computer), el set de instrucciones y el bus de memoria de programa pueden diseñarse de tal manera que todas las instrucciones tengan una sola posición de memoria de programa de longitud.

Además, al ser los buses independientes, la CPU puede acceder a los datos para completar la ejecución de una instrucción, y al mismo tiempo leer la siguiente instrucción a ejecutar.

Ventajas de esta arquitectura:

- El tamaño de las instrucciones no está relacionado con el de los datos, y por lo tanto puede ser optimizado para que cualquier instrucción ocupe una sola posición de memoria de programa, logrando así mayor velocidad y menor longitud de programa.
- El tiempo de acceso a las instrucciones puede superponerse con el de los datos, logrando una mayor velocidad en cada operación.

Tipos de memoria

En un sistema abierto como un PC, los programas se encuentran en la memoria RAM (lectura/escritura) para posibilitar la carga de uno u otro programa desde, por ejemplo el disco duro, según lo desee el usuario.

La memoria RAM (Ramdom Access Memory, memoria de acceso aleatorio) es un tipo de memoria que permite tanto la lectura (read) como la escritura (write). Es volátil, esto es, los datos se pierden si deja de alimentarse.

Cuando el sistema necesita leer un dato de la memoria RAM, lo primero que hace es indicar la posición de memoria de donde desea obtener el dato. Dependiendo de si la operación que va efectuar es una lectura o una escritura, se pone a uno o a cero una señal de lectura escritura (Read=1/Write=0). Si la operación es de lectura, la memoria entrega el dato deseado. Por el contario, si es de escritura, el dato se almacena en la memoria.

Sin embargo en un microcontrolador la memoria de programa puede ser:

- OTP
- ROM
- PROM
- EPROM
- EEPROM
- Flash

Memoria ROM: Su nombre corresponde a las iniciales de Read Only Memory, que significa memoria de sólo lectura. Al contrario que la memoria RAM, este tipo de memoria solo permite la lectura (read), sin embargo esta memoria también es RAM debido a que puede leerse cualquier posición de memoria en el mismo tiempo.

En un sistema abierto como un PC su función es contener los datos y programas de arranque, para que el microprocesador pueda comunicarse con el resto del sistema. En este caso se le denomina BIOS (Basic Input/Output System), porque contiene las instrucciones básicas de entrada y salida. Su funcionamiento es idéntico al de las memorias RAM, con la excepción de que al ser únicamente de lectura, no precisa de la señal read/write.

Memoria OTP: La memoria de programa, que puede ser una EPROM, no dispone de ventana para el borrado por lo que sólo se puede programar una vez

Por último conveniente recordar que en un sistema abierto aunque la Memoria Central es un elemento fundamental no está dentro del microprocesador, pero sí está dentro de un microcontrolador.

Unidad de entradas/salidas (interfaz)

Esta unidad comunica al sistema con el mundo exterior, permitiendo la introducción y la extracción de información al sistema. Estas unidades consisten generalmente en registros que, accionados por los buses de control y direcciones, almacenan la información suministrada por el bus de datos.

Estos registros son accesibles desde el exterior por una serie de terminales para su conexión a cualquier dispositivo que se deba accionar.

En las entradas se introduce la información de manera que cuando se ordena, mediante los buses de control y de direcciones, el sistema selecciona en un instante dado cuales de esas informaciones presentes en las entradas deben transferirse al bus de datos para su proceso.

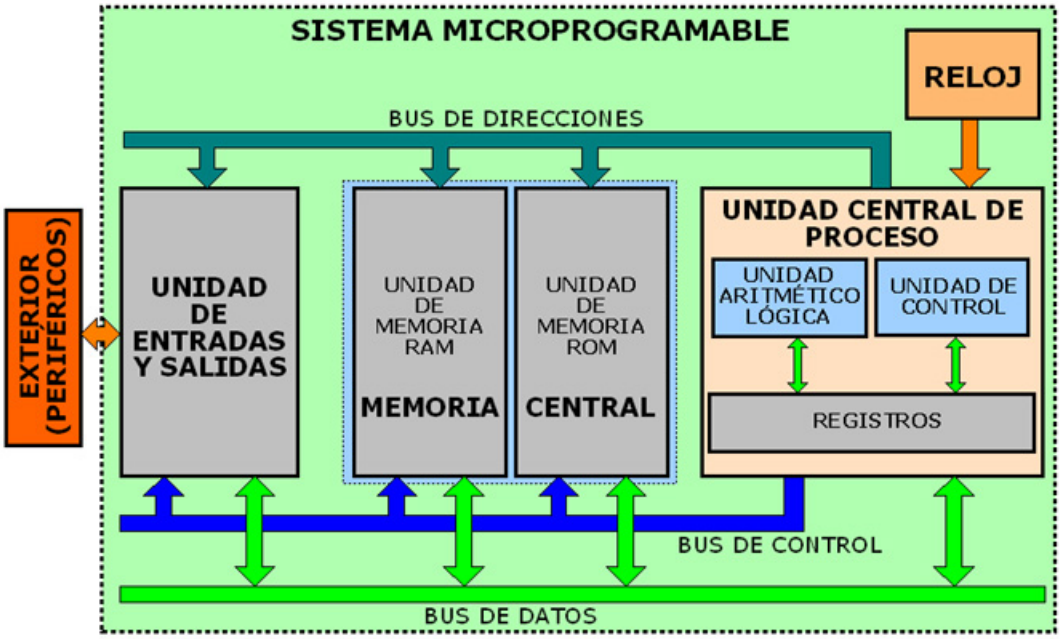
La parte de salida, por el contrario, consiste en una serie de registros en los que el sistema deposita el resultado de la información ya procesada.

Nota: Al igual que sucede en un sistema abierto con la Memoria Central, la Unidad de E/S es un elemento que no está en el interior del microprocesador, pero si dentro de un microcontrolador.

Buses del sistema

Los elementos de un sistema basado en CPU están interconectados entre sí mediante "canales" de información o buses. Estos buses están compuestos por diferentes líneas (hilos eléctricos) que transportan información del mismo tipo. El número de líneas que compone el bus indica el ancho del bus. Por ejemplo, si tenemos un bus de datos de 16 bits, significa que circulan 16 bits en paralelo a la vez, por lo que el bus tiene un ancho de 16 líneas. En los sistemas microprogramables, existen tres buses fundamentales que son:

- Bus de direcciones (address bus)
- Bus de datos (data bus)
- Bus de control (Control Bus)



Bus de direcciones (address bus)

Por este bus van a circular los bits (combinación binaria) que seleccionaran la posición de la memoria o el registro de entrada/salida en el que deseamos leer o escribir. Este bus tiene tantas líneas (conductores) como bits de dirección tenga el sistema. El número de líneas que tiene un sistema basado en microprocesador depende del fabricante y determina la cantidad máxima de memoria que puede direccionar el sistema. El número total de direcciones de memoria que pueden direccionarse viene dado por la formula:

N° Direcciones de memoria=$2^{\text{N° Líneas del bus de direcciones}}$

Por ejemplo, el microprocesador 8086 de Intel poseía un bus de direcciones de 20 líneas, por tanto:

N° Direcciones de memoria= 2^{20}= 1.048.576 posiciones de memoria.

Si en cada posición de memoria se almacena 1 byte (8 bits), el microprocesador 8086 podía direccionar como máximo una memoria de 1.024 Kbytes=1 Mbyte.

Como se puede observar en la figura, este bus es unidireccional. Es decir, las líneas van en un único sentido, siendo la CPU la única que puede poner datos en el.

Habitualmente, las líneas del bus de direcciones se denominan con la letra inicial A (de Address), por ejemplo: A15 (bit de mayor peso) A14 ... A1 A0 (bit de menor peso)

Bus de datos (data bus)

Por este bus circularan los bits que componen la información binaria, ya sean instrucciones o datos contenidos en la posición de memoria o en los registros de entrada/salida, seleccionada por el bus de direcciones. Al igual que el bus de direcciones, este tiene tantos hilos como bits tenga la palabra de datos con que opera el sistema.

Por ejemplo, un microprocesador de 8 bits posee un bus de datos formado por 8 hilos. Como podemos observar en la figura, este bus es bidireccional, es decir, que puede ir en ambos sentidos. Esto es, porque los dispositivos pueden recibir información (escritura) o enviarla (lectura), según el estado de la línea read/write del bus de control. Habitualmente, las líneas del bus de datos se denominan con la letra inicial D (de data), por ejemplo: D7 (bit de mayor peso) D6 ... D1 D0 (bit de menor peso)

Bus de control (Control Bus)

Este bus está formado por una serie de líneas denominadas líneas de control, por las que va a circular el conjunto de señales necesarias para la correcta coordinación de todos los elementos del sistema, tales como: órdenes de lectura o escritura, inhabilitación (desactivación) de un dispositivo, etc.

Funcionamiento de los buses

Cuando uno de los registros de entrada y salida recibe su dirección por el bus de direcciones, el dispositivo se activa y según la orden de leer o escribir presente en el bus de control, pasan la información que contienen al bus de datos, o se cargan con la información que contiene el bus de datos.

Por ejemplo, si deseamos enviar un dato por el puerto paralelo de un PC, lo primero que haremos será poner la dirección de ese puerto en el bus de direcciones. Como lo que queremos es enviar, es decir, escribir, activaremos en el bus de control la señal de escritura y, por último, enviaremos por el bus de datos la información a sacar por dicho puerto. Si, por el contrario, queremos leer un dato por el puerto paralelo, primero enviaremos por el bus de direcciones la dirección donde se encuentra ese dispositivo. Como deseamos leer su contenido, activaremos la señal de lectura del bus de control y, por último, el controlador de ese dispositivo nos pondrá en el bus de datos su contenido.

Software de un sistema basado en CPU

El software que se escribe y que posteriormente el sistema microprogramable ejecuta está formado por una serie de instrucciones que indican al sistema lo que debe realizar en cada momento.

Hasta ahora hemos hablado de que la CPU recoge las instrucciones de la memoria. ¿Pero cómo están compuestas estas instrucciones? Las instrucciones, como su propio nombre indica, van a decir que se debe hacer con los datos, o con que datos se debe realizar cierta operación.

Formato de una instrucción

Las instrucciones están formadas por dos partes, tal y como muestra a continuación

INSTRUCCIÓN	
CÓDIGO OPERACIÓN	OPERANDO

- **Código de operación:** Este indica a la CPU que tipo de operación debe realizarse, como por ejemplo, una suma, una resta, comparación u operación lógica.
- **Operando:** Por el contrario, este indica con que datos se deben realizar dichas operaciones, o bien, las direcciones de donde se deben obtener dichos datos.

El código de operación, está siempre presente en cualquier tipo de instrucción. Por el contrario, el operando puede o no estarlo. Por ejemplo, para que la CPU detenga la ejecución de un programa, es suficiente con el código de operación. Sin embargo, para realizar una suma, a parte del código de operación habrá que indicar con que dato se debe sumar, ya sea explícitamente después del código de operación o indicando en su lugar la posición de memoria donde se encuentra dicho dato.

Las instrucciones se pueden clasificar según el tipo de operación que realicen en:

- **Instrucciones lógicas:** Realizan las operaciones lógicas entre operandos, tales como suma lógica, complementación, etc.
- **Instrucciones aritméticas:** Realizan las operaciones aritméticas, tales como la suma aritmética, división aritmética, etc. Este tipo de operaciones se realiza entre acumuladores y posiciones de memoria.
- **Instrucciones de salto:** Este tipo de instrucciones corresponde a los saltos condicionales (hacen una cosa u otra según el resultado de ciertas operaciones) e incondicionales (saltan a cierto lugar en el programa).
- **Instrucciones de transferencia de datos (entrada/ salida):** Estas instrucciones "ordenan" el paso de información entre distintos dispositivos del sistema.
- **Instrucciones de control:** Controlan el desarrollo del programa, lo finaliza, inicializa, no operación (NOP), etc.

Lenguajes de programación para sistemas basados en CPU

Para introducir los programas y datos en un sistema microprogramable, necesitamos de un método que sea comprensible tanto para el usuario como para el sistema. Sin embargo, no existe un lenguaje que sea igualmente comprensible para el usuario como para el sistema. Mientras que el primero utiliza un lenguaje complejo, el sistema únicamente opera en lenguaje binario.

Este lenguaje binario que utiliza la máquina es muy complejo de entender por su abstracción y, por tanto, de programar. Por esta razón se clasifican los lenguajes en niveles según estén más o menos cercanos al lenguaje de la máquina en:

- Lenguaje de bajo nivel o código máquina.
- Lenguaje ensamblador.
- Lenguaje de alto nivel.

Lenguaje de bajo nivel o código máquina

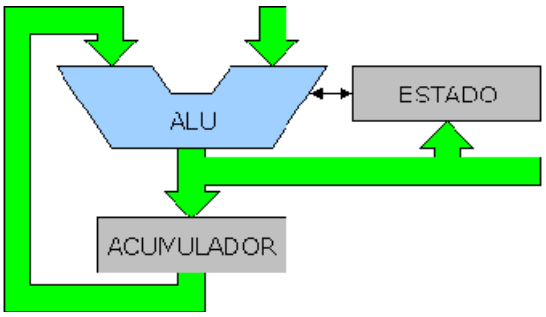
Es difícilmente entendible para el usuario, porque es el que ejecuta directamente la CPU. Este lenguaje está formado por palabras binarias, cuya longitud depende del sistema empleado.

Por ejemplo, una CPU de 8 bits utilizara palabras formadas por 8 bits.

El problema de este lenguaje es que cada CPU utiliza sus propios códigos máquina, por lo que los programas no son intercambiables entre los sistemas microprogramables. Aunque la máquina solo entiende códigos binarios (ceros y unos), este tipo de lenguaje se puede programar en hexadecimal, que es más manejable y su conversión al formato binario es directa.

En tabla siguiente se da un ejemplo de código de operación de instrucciones para un microcontrolador de la Familia PIC16F88, que utiliza una arquitectura Harvard y donde la memoria de programa tiene una longitud de 14 bits y la de datos de 8 bits.

Estas instrucciones suman dos valores (7+8) y el resultado lo depositan en la dirección de memoria de datos 10 h.



Binario	Hexa	Código	Operando	Instrucción
11000000000111	3007	1100xx	0000 0111	Carga en el acumulador el dato indicado en el operando
11111000001000	3E08	11111x	0000 1000	Suma el acumulador y el dato indicado en el operando y guarda el resultado en el acumulador
00000010010000	0090	0000001	0010000	Almacena el resultado contenido en el acumulador en la dirección indicada

Nota: "*x*" representa un valor indiferente, pero se ha tomado en binario como en hexadecimal como 0, tal y como lo hace el ensamblador MPLAB de microchip.

Este formato binario, denominado lenguaje ó código máquina, puesto que es el que realmente entiende el sistema, es el que debe estar en la memoria de programa para que pueda funcionar. Se comprende, no obstante, que escribir un programa, que puede constar de cientos, miles o millones de instrucciones, a este nivel, seria extraordinariamente penoso, sujeto a errores y muy difícil de interpretar por una persona, una vez escrito. Sin embargo, la velocidad con que se ejecuta es muy elevada, ya que las instrucciones son ejecutadas directamente por la máquina. Otra ventaja de este tipo de lenguaje reside en que nos podemos meter en lo más profundo de la máquina.

Lenguaje ensamblador

El lenguaje máquina no es muy manejable por su elevada complejidad, por lo que, para facilitar la confección de programas se utiliza otro tipo de lenguaje denominado lenguaje ensamblador o simbólico.

En el lenguaje ensamblador, las instrucciones se representan por nemónicos o combinaciones de letras que recuerdan el significado de la instrucción en inglés. Así, por ejemplo, en la siguiente tabla se representan los nemónicos utilizados para las instrucciones anteriores.

Mnemónico	Hexa	Código	Operando	Instrucción
movlw 0x07	3007	1100xx	0000 0111	Carga en el acumulador el dato indicado en el operando
addlw 0x08	3E08	11111x	0000 1000	Suma el acumulador y el dato indicado en el operando y guarda el resultado en el acumulador
movwf 0x10	0090	0000001	001 0000	Almacena el resultado contenido en el acumulador en la dirección indicada

Además, las direcciones en vez de figurar con su valor binario real, pueden figuran bajo nombres simbólicos (etiquetas) que se pueden asignar de modo que recuerden su significado o la dirección de las instrucciones.

La estructura de una instrucción escrita en lenguaje ensamblador constaría en general de tres campos:

- **Etiqueta**, o nombre simbólico asignado a la dirección donde se encuentra esa instrucción, y que servirá como operando a otras instrucciones de bifurcación o salto situadas en otro punto del programa.
- **Instrucción**, mnemónico equivalente a la primera palabra de datos de la instrucción en lenguaje máquina.
- **Operando**, o nombre simbólico asignado a la dirección en que se encuentran los datos.

Como ejemplo, a continuación se representa una porción de programa redactado para los microcontroladores PIC16 y su equivalencia en lenguaje máquina.

Ensamblador			Lenguaje máquina	
Etiqueta	***Mnemónico***	***Operando***	***PC***	***Código+Operando***
INICIO	movlw	0x07	000000	3007 h
	addlw	0x08	000001	3E08 h
	movwf	RESULTADO	000002	0090 h

Notas:
PC: Dirección de la memoria de programa (program counter).
RESULTADO: El ensamblador permite sustituir valores numéricos por etiquetas así RESULTADO será igual a 10 h.
Se comprende que, con un poco de práctica, el programa en ensamblador es mucho más fácil de escribir y leer que el lenguaje máquina.
Dependiendo del tipo de CPU las direcciones se especifican de diversas maneras (absoluta, directa, segmentada, como primer operando el byte bajo y como segundo el byte alto, etc.).

Programa ensamblador

Una vez escrito el programa en lenguaje ensamblador, su conversión a lenguaje máquina puede hacerse automáticamente en una PC mediante un programa, al cual habrá que indicarle, entre otras cosas, la equivalencia numérica de los símbolos o etiquetas utilizados para representar valores numéricos como operandos (en el ejemplo anterior, **RESULTADO**), mientras que las equivalencias numéricas de las etiquetas con la posición de cada instrucción en el programa ensamblador se generarán automáticamente (en el ejemplo anterior, **INICIO**).

Este proceso de traducción a lenguaje máquina se conoce como "ensamblado" y el programa de ordenador que lo realiza, ensamblador.

Lenguajes de alto nivel

Es el lenguaje más cercano al usuario y, por tanto, el más evolucionado, ya que no se basa en la arquitectura de la máquina, permitiendo utilizar el mismo programa en diferentes dispositivos.

El nombre de las instrucciones y sentencias se corresponde con el nombre en ingles de la tarea que realizan, dependiendo ahora el repertorio de instrucciones no de la CPU, sino del paquete software con el que trabaja o del sistema operativo.

Algunos ejemplos de lenguajes de alto nivel son:

- BASIC
- C
- JAVA

Los lenguajes de alto nivel son muy parecidos al lenguaje del usuario pero muy distinto del que comprende la máquina. Por tanto, para ejecutarse debe ser convertido y la conversión suele introducir bastante más código que si el programa se escribiese directamente en ensamblador por lo su ejecución es más lenta. Además el programa se hace más largo.

El inconveniente de este tipo de lenguaje es que no podemos meternos en lo más profundo de la máquina. Sin embargo se puede escribir la mayor parte de un programa en un lenguaje de alto nivel, y luego añadir subrutinas realizadas en lenguaje máquina.

Proceso de programación

Una vez escrito el programa (programa fuente) en uno de los anteriores lenguajes, este debe "transformarse" en código máquina que es lo único que entiende el sistema microprogramable.

Para ello, disponemos de las siguientes herramientas:

- **Ensambladores:** Se utilizan para transformar el lenguaje ensamblador a código máquina.
- **Compiladores:** Se emplean para transformar el resto de los lenguajes a código máquina.